

An Empirical Analysis of Technical Debt in Open-Source Machine Learning Repositories

Vidhumol Pandippilly Antony | ST86889 | Supervisor: Prof. Dr. sc. ing. Boriss Mišņevs | Computer Sciences 45483 | Riga, 2026

1. INTRODUCTION

Background

Machine Learning systems introduce engineering challenges distinct from conventional software. Developers frequently admit implementation shortcuts in source code — known as Self-Admitted Technical Debt (SATD).

Research Gaps Addressed

- No prior study examined whether ML sub-categories accumulate debt differently
- No LLM-based semantic SATD classification with severity grading existed
- All prior work scanned Python source files only
- No SDLC phase-level analysis of technical debt accumulation

Research Questions

- RQ1: Are there statistically significant differences in the amount of technical debt in different ML sub-categories?
- RQ2: What types of new debt appear in MLOps and LLM repositories that have not existed before?
- RQ3: Are python-exclusive SATD analysis underestimates total debt in ML projects?
- RQ4: To what extent can a composite model predict debt level as well as single-signal approaches?
- RQ5: When the richer multi-modal approach to investigating debt patterns is applied, are there differences in the patterns of ML-specific debt patterns as compared to traditional software?

Study Scope

526 open-source repositories (446 ML + 80 Non-ML control) across 5 ML sub-categories: Deep Learning, NLP, Computer Vision, MLOps, AutoML/RL

KEY STATISTICS

526

Repositories
Analysed

7,753

SATD Instances
Detected

2,803

LLM Classified
Comments

12

Notebooks
Pipeline

KEY REFERENCES

- Bhatia et al. (2025) — Empirical study of SATD in ML software. ACM TOSEM.
- Selvanayagam et al. (2026) — SATD in LLM vs ML software. arXiv:2601.06266.
- Pepe et al. (2024) — Taxonomy of SATD in Deep Learning systems. arXiv:2409.11826.
- Sculley et al. (2015) — Hidden technical debt in ML systems. NeurIPS.
- Bavota & Russo (2016) — Large-scale empirical study on SATD. MSR.
- Ximenes et al. (2025) — Code technical debt in ML systems. CAIN 2025.

2. METHODOLOGY

12-Notebook Analytical Pipeline

Baseline Phase (NB01–05)

- NB01: Repository collection — 526 repos via async GitHub API
- NB02: SATD extraction — 10 keywords, 7,458 Python SATD records
- NB03: Static analysis — Radon (CC, MI) + Flake8 PEP8 violations
- NB04: Commit history — 500 most recent commits per repository
- NB05: Baseline ML classifier — Random Forest + Logistic Regression

Innovation Phase (NB06–09)

- NB06: LLM classifier — Llama 3.1 (Groq API), 19-category taxonomy
- NB07: K-means clustering — TF-IDF vectorisation, k=8 optimal
- NB08: Multi-file scan — YAML, Jupyter, dependency & docs
- NB09: Statistical validation — Kruskal-Wallis, Chi-Square, Spearman

Extension Phase (NB10–12)

- NB10–11: Replicate full pipeline on 427 new repositories
- NB12: Combined 526-repo validation + SDLC phase analysis

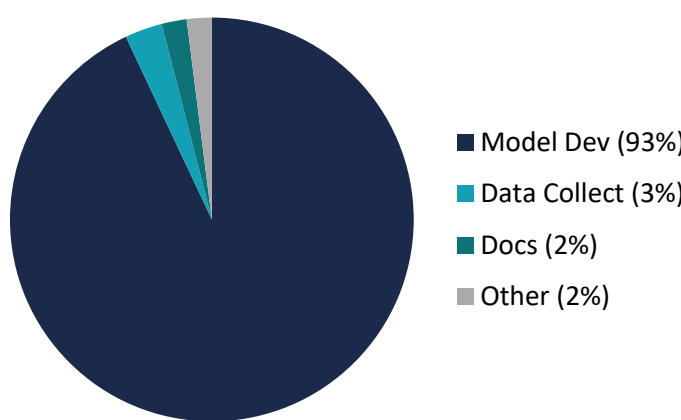
6 Novel ML-Specific Debt Types Introduced:

MLOps Pipeline Debt • Experiment Tracking Debt • Data Versioning Debt • Monitoring & Observability Debt • Prompt Template Debt • Token Budget Debt

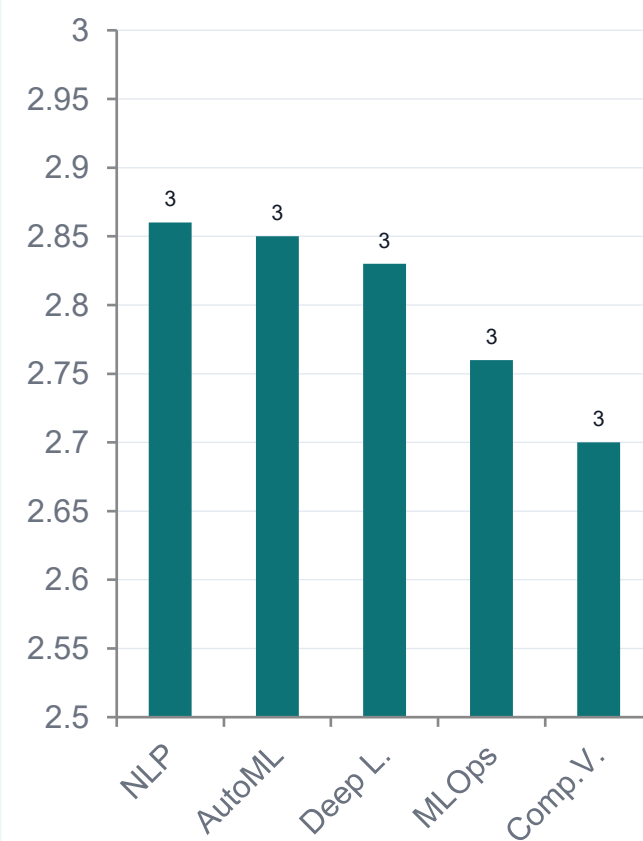
4. STATISTICAL VALIDATION RESULTS

Statistical Test	Result	Interpretation
Kruskal-Wallis H-test	H=25.26, p=0.000045 ✓	SATD volume differs significantly across ML sub-categories
Chi-Square (RQ2)	$\chi^2=1204.66$, p=0 ✓	Debt composition is domain-specific
Dunn Post-Hoc	MLOps vs NLP: p=0.0003	MLOps is the statistical outlier
Spearman — Commits	rho=0.369, p<0.0001 ✓	Commit activity = strongest SATD predictor
Spearman — Age	rho=0.054, p=0.213 ✗	Age alone does not predict SATD

SATD by SDLC Phase



Avg Severity (1–5)



CONTACT INFORMATION

Vidhumol Pandippilly Antony

LinkedIn: www.linkedin.com/in/vidhumol-pandippilly-antony
GitHub: github.com/vidhumol/ml-technical-debt-analysis
Transport and Telecommunication Institute, Riga, Latvia, 2026

3. RESULTS & DISCUSSION

Debt Category Distribution (2,803 LLM Classified)

Code Debt dominates at 70.9% (1,989 instances), contradicting prior work showing Requirement Debt as most common. LLM semantic reclassification reveals that many TODO comments are code quality issues, not feature requests.

Non-Python SATD Findings

- Python-only analysis underestimates by 4.1% (257 missed instances)
- Deep Learning: largest gap at 6.2% (Jupyter notebook debt)
- MLOps: 5.3% gap (YAML pipeline and Dockerfile debt)
- Documentation files dominate non-Python SATD (162/295 = 55%)

8 Unsupervised Behavioural Debt Clusters (K-Means, k=8)

- Code Quality Suppression (1,478) — universal across all ML categories
- Path Management Debt (276) — near-exclusive MLOps pattern ★ NEW**
- Import Suppression Debt (187) — NLP dominant ★ NEW**
- Type Error Workaround (127) — 126/127 instances from Deep Learning

SDLC Phase Analysis (Novel Contribution)

- 93% of SATD accumulates in P3 Model Development phase
- CI/CD and Documentation debt ONLY visible via non-Python scanning
- MLOps: only category with P6 Monitoring & Operations debt

DEBT CATEGORY BREAKDOWN (LLM CLASSIFIED)



5. CONCLUSIONS / OUTCOMES

Four Original Contributions:

- First statistically validated SATD comparison across ML sub-categories ($\chi^2=1204.66$, $p\approx 0$)
- First LLM-based SATD classifier with severity grading — 6 novel debt types
- First evidence of non-Python debt — 4.1% underestimation confirmed
- First SDLC phase-level SATD study in ML repositories

Practical Implication:

ML practitioners should prioritise debt detection in model training scripts, YAML pipeline files, and Jupyter notebooks — not Python source code alone.



TRANSPORT AND
TELECOMMUNICATION
INSTITUTE